

UNIT 3: CPU SCHEDULING & DEADLOCK

AKTU BCS401 • 3-Page Rapid Recall Cheatsheet

PAGE 1 / 3 • CONCEPTS & SCHEDULERS

1. PROCESS CONCEPTS & VIRTUAL ADDRESS SPACE

Program vs Process

- **Program:** Passive entity; static machine binary code on disk.
- **Process:** Active entity; Program in execution in RAM with PCB & registers.
- **CPU-I/O Burst:** Execution alternates: Compute (CPU) ↔ Wait (I/O).

Process Address Space Layout

- **Text:** Compiled instructions (Read-Only).
- **Data:** Global/static variables (.data init, .bss uninit).
- **Heap:** Dynamic runtime memory (malloc, new) → Grows UP.
- **Stack:** Local vars, activation frames → Grows DOWN.

2. PROCESS STATES & PCB (PROCESS CONTROL BLOCK)

5 & 7 States Life Cycle Model

- **NEW → READY:** Admitted by Long-Term Scheduler (LTS).
- **READY → RUNNING:** Dispatched by Short-Term Scheduler (STS).
- **RUNNING → READY:** Preempted via timer interrupt/quantum.
- **RUNNING → WAITING:** I/O request or event wait syscall.
- **WAITING → READY:** I/O completion interrupt.
- **SUSPENDED (Ready/Blocked):** Swapped to disk by MTS.

PCB Fields (task_struct in Linux)

- **PID / PPID:** Unique integer process / parent ID.
- **Process State:** Running, Interruptible, Zombie.
- **Program Counter (PC):** Next instruction memory pointer.
- **CPU Registers:** Saved hardware context on preemption.
- **Scheduling Info:** Priority, nice value, queue pointers.
- **Memory & I/O:** Page tables, limits, open file descriptors.

3. SCHEDULERS HIERARCHY & DISPATCHER LATENCY

Scheduler	Alternate Name	Primary Function	Speed / Frequency	Multiprogramming Control
Long-Term (LTS)	Job Scheduler	Pool (Disk) → Ready Queue (RAM)	Slow (Minutes)	Controls degree of multiprogramming
Short-Term (STS)	CPU Scheduler	Ready Queue → CPU Execution	Ultra-fast (Milliseconds)	No direct control over degree
Medium-Term (MTS)	Swapper	RAM ↔ Disk Swap Space	Medium (Seconds)	Reduces degree during thrashing

Context Switch Overhead & Efficiency: $\text{CPU Efficiency } (\eta) = \text{Useful Time} / (\text{Useful Time} + \text{Overhead}) * 100\%$

4. THREADS & MULTITHREADING MODELS

Thread Components

Shared: Code, Data, Heap, Open files, OS signals.
Private: Thread ID (TID), PC, Registers, Call Stack.

ULT vs KLT

ULT: Fast user-space switch, 1 block blocks all, no multicore.
KLT: Kernel managed, true multicore, independent blocking.

Thread Models

- **M:1:** User library, low kernel cost.
- **1:1:** High concurrency (Linux/Win).
- **M:N:** Hybrid multiplexing (Go).

5. GOLDEN SCHEDULING PERFORMANCE CRITERIA

Metric	Formula	Objective	Exam / Practical Significance
Turnaround Time (TAT)	$\text{TAT} = \text{Completion Time (CT)} - \text{Arrival Time (AT)}$	MINIMIZE	Total time spent by process inside system.
Waiting Time (WT)	$\text{WT} = \text{Turnaround Time (TAT)} - \text{Burst Time (BT)}$	MINIMIZE	Time spent waiting in Ready Queue. Standard benchmark.
Response Time (RT)	$\text{RT} = \text{First CPU Time} - \text{Arrival Time (AT)}$	MINIMIZE	Delay to first reaction. Non-preemptive me $\text{RT} == \text{WT}$.
Throughput	$\text{Completed Processes} / \text{Total Time}$	MAXIMIZE	Higher is better for overall computational productivity.
CPU Utilization	$(\text{Active CPU Time} / \text{Total Time}) * 100\%$	MAXIMIZE	Ideal target range: 40% (light) to 90% (heavy load).

1. FCFS & THE CONVOY EFFECT

- **Mechanism:** Simple FIFO queue. Non-preemptive. Jo pehle arrive hua, wo pehle run karega.
- **Convoy Effect:** Ek bada CPU-bound process running rehta hai aur kayi chhote I/O-bound processes queue me block rehte hain. I/O devices idle rehte hain aur Avg WT drastically badh jata hai.
- **Numerical Proof:** P1(24), P2(3), P3(3) → Order P1,P2,P3 gives Avg WT = **17.0 ms** vs Order P2,P3,P1 gives Avg WT = **3.0 ms** (82.3% faster!).

2. SJF (NON-PREEMPTIVE) & SRTF (PREEMPTIVE SJF)

SJF (Shortest Job First)

- **Optimality:** Provably *optimal* for minimum Avg WT.
- **Limitation:** Future burst time unknown in advance. Long jobs face **Starvation**.

Exponential Smoothing: $\tau_{n+1} = \alpha \cdot \tau_n + (1-\alpha) \cdot \tau_n$

SRTF (Shortest Remaining Time First)

- **Preemption:** Naye process ke arrival par Remaining Time compare hota hai. If $BT_{new} < Rem_{curr}$ → Preempt!
- **Characteristics:** Even lower Avg WT than SJF. Frequent context switches increase CPU switching overhead.

3. PRIORITY SCHEDULING & AGING TECHNIQUE

Priority Scheduling

- **Convention:** Typically **Smaller Integer = Higher Priority** (0 = Top).
- **Preemptive vs Non-Preemptive:** Higher priority incoming preempts running.
- **Starvation:** Continuous stream of high priority jobs denies CPU to low priority.

Aging Technique

- **Solution:** Gradually increases priority of waiting processes as time progresses.
- **Formula / Rule:** Increase priority by 1 every Δt (e.g. 15 mins).
- **Guaranteed prevention of indefinite blocking.**

4. ROUND ROBIN (RR) & QUANTUM SIZING DYNAMICS

- **Mechanism:** Ready Queue is Circular FIFO. Har process ko **Time Quantum 'q'** milta hai. Quantum khatam hote hi timer interrupt process ko tail par bhej deta hai.
- **q → ∞ (Very Large):** Degenerates into FCFS (Convoy effect returns; poor interactive response).
- **q → 0 (Very Small):** Processor sharing, but massive Context Switch Overhead thrashing.
- **80% Rule of Thumb:** At least 80% of CPU bursts should be shorter than time quantum 'q'. Typical: 10ms - 100ms.

5. COMPREHENSIVE NUMERICAL COMPARISON TABLE

Algorithm	Preemptive?	Optimal Metric?	Starvation Risk?	Best Used In
FCFS	No	None	No	Batch queues, print spoolers
SJF	No	Minimum Avg WT	Yes (Long jobs)	Long-term batch scheduling
SRTF	Yes	Absolute Minimum Avg WT	Yes (Long jobs)	Short-job optimized systems
Priority	Both	Priority compliance	Yes (Low priority)	Real-time and mission-critical OS
Round Robin	Yes	Best Response Time (RT)	No (Starvation Free)	Time-sharing interactive systems

6. MULTILEVEL QUEUES & MULTIPROCESSOR SCHEDULING

MLQ vs MLFQ

- **MLQ:** Static queue partitions (System, Interactive, Batch). No movement. High starvation risk.
- **MLFQ:** Dynamic movement. Jobs demoted on quantum expiry, promoted via Aging. Self-tuning!

Multiprocessor & SMP

- **SMP:** Peers, self-scheduling, per-core queues.
- **Processor Affinity:** Process stays on same CPU for warm cache (L1/L2 hits).
- **Load Balancing:** Push & Pull migration across cores.

1. DEADLOCK DEFINITION & THE 4 NECESSARY COFFMAN CONDITIONS

- **Definition:** Permanent blocking of a set of processes where each holds a resource and waits for another.
- **Deadlock occurs IF AND ONLY IF all 4 conditions hold simultaneously:**
 1. **Mutual Exclusion:** At least 1 non-shareable resource. Only 1 process at a time can use.
 2. **Hold and Wait:** Process holds 1+ resources while waiting to acquire additional held resources.
 3. **No Preemption:** Resources cannot be forcibly revoked; released only voluntarily.
 4. **Circular Wait:** Closed cyclic dependency chain: $P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_n \rightarrow P_0$.

2. RESOURCE ALLOCATION GRAPH (RAG) & CYCLE RULES

- RAG Components**
- **Vertices:** Processes P (Circles), Resources R (Rectangles with dots).
 - **Request Edge:** $P_i \rightarrow R_j$ (Process waits for resource).
 - **Assignment Edge:** $R_j \rightarrow P_i$ (Resource allocated to process).

- Golden Cycle Theorems**
- **NO Cycle:** 100% Guaranteed **NO DEADLOCK**.
 - **Single-Instance + Cycle:** **GUARANTEED DEADLOCK**.
 - **Multi-Instance + Cycle:** **MAY OR MAY NOT** have deadlock (External processes can break cycle!).

3. DEADLOCK HANDLING STRATEGIES & PREVENTION

Strategy	Methodology	Core Technique
1. Ignorance	Ostrich Algorithm (Ignore it)	Used by Linux, Windows (Deadlocks assumed rare; avoidance too costly).
2. Prevention	Invalidate at least 1 Coffman condition	Havender's Protocol: Strict linear ordering $F: R \rightarrow N$. Request only if $F(R_i) < F(R_j)$. Cycles mathematically impossible!
3. Avoidance	Dynamic safe-state checking	Banker's Algorithm: System never enters Unsafe state.
4. Detection	Periodic graph analysis & recovery	Wait-For Graph (WFG) cycle detection $O(V^2)$ + Process Abort / Preemption.

4. DEADLOCK AVOIDANCE: BANKER'S ALGORITHM MASTERCLASS

Core Matrix Formula: $Need[i][j] = Max[i][j] - Allocation[i][j]$

- **Safety Algorithm Steps:**
 1. Set **Work** = Available and **Finish[i] = False** for all i.
 2. Find process P_i where **Finish[i] == False** AND **Need_i ≤ Work**. If none found → Step 4.
 3. **Work = Work + Allocation_i**, **Finish[i] = True**. Repeat Step 2.
 4. If all **Finish[i] == True** → **SAFE STATE!** Safe Sequence exists.
- **Resource Request Algorithm:** 1) $Req \leq Need$; 2) $Req \leq Available$; 3) Pretend allocate & run Safety check.

5. DEADLOCK DETECTION & RECOVERY PROTOCOLS

- Detection & Wait-For Graph (WFG)**
- Single instance resources: Collapse resource nodes → WFG.
 - Cycle detection via DFS in $O(V^2)$ time indicates deadlock.
 - Multiple instance: Detection matrix algorithm using Request matrix.

- Recovery Mechanisms**
- **Process Abort:** Abort all vs Abort one-by-one by cost.
 - **Resource Preemption:** Pick victim, Rollback to checkpoint.
 - **Starvation Fix:** Include rollback count in victim cost.